DHBW
Duale Hochschule
Baden-Württemberg

18. Oktober 2026

Verteilte Systeme

02 + 03 Kommunikation in verteilten Systemen

Robin Windey // Kevin Kiefer

Agenda

Teil 1

1. Grundlagen: Kommunikation in verteilten Systemen
2. HTTP & REST-APIs
3. HTTPS & Authentifizierung

Teil 2

1. MQTT - Publish/Subscribe
2. Ausblick: Weitere Protokolle

Kommunikation in verteilten Systemen

Warum Kommunikation kritisch ist

- Komponenten laufen auf verschiedenen Systemen
- Netzwerklatenz und -fehler sind unvermeidbar
- Unterschiedliche Programmiersprachen und Plattformen
- Sicherheit und Authentifizierung erforderlich

Kommunikation in verteilten Systemen

Kommunikationsmodelle

Request/Response (Anfrage/Antwort)

- 1:1 Kommunikation
- Client initiiert, Server antwortet
- Beispiele: HTTP (REST, SOAP, …) , gRPC

Publish/Subscribe (Veröffentlichen/Abonnieren)

- 1:n oder n:m Kommunikation möglich
- Entkopplung von Sender und Empfänger
- Beispiele: MQTT, Kafka, RabbitMQ

HTTP

Definition Hypertext Transfer Protocol

- Anwendungsprotokoll für Web-Kommunikation
- Läuft über TCP/IP (Port 80) oder HTTPS (Port 443)
- Zustandslos (stateless)
- Text-basiert und menschenlesbar

HTTP

HTTP-Methoden (Teil der Client-Anfrage)

Methode	Bedeutung	Verwendung
GET	Daten abrufen	Ressource lesen (idempotent)
POST	Daten senden	Neue Ressource erstellen
PUT	Daten aktualisieren	Ressource ersetzen (idempotent)
PATCH	Teilupdate	Ressource teilweise ändern
DELETE	Daten löschen	Ressource entfernen (idempotent)

Was bedeutet "idempotent"? => Mehrfaches Ausführen der gleichen Anfrage hat denselben Effekt wie einmaliges Ausführen
Beispiel GET: 10x abrufen = 1x abrufen (keine Änderung)
Beispiel PUT: 10x aktualisieren = 1x aktualisieren (gleiches Ergebnis)
POST ist NICHT idempotent: 10x erstellen → 10 neue Ressourcen

* Siehe auch https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods

HTTP

HTTP-Statuscodes (Teil der Server-Antwort)

- **2xx – Erfolg**
  - 200 OK
  - 201 Created
  - 204 No Content
- **4xx - Client-Fehler**
  - 400 Bad Request
  - 401 Unauthorized
  - 403 Forbidden
  - 404 Not Found
- **5xx - Server-Fehler**
  - 500 Internal Server Error
  - 503 Service Unavailable

Beispiel: HTTP-Antwort (Response)

Aufbau einer typischen HTTP-Response

```
HTTP/1.1 200 OK
Date: Sat, 11 Oct 2025 10:30:00 GMT
Server: nginx/1.21.0
Content-Type: application/json; charset=utf-8
Content-Length: 95
Connection: keep-alive

{
  "id": 123,
  "name": "Max Mustermann",
  "email": "max@example.com",
  "status": "active"
}
```

Komponenten

1. **Statuszeile:** HTTP/1.1 200 OK (Protokoll + Statuscode)
2. **HTTP-Header:** Metadaten über die Nachricht (siehe nachfolgende Folien)
3. **Response Body:** Die eigentlichen Nutzdaten (hier: JSON)

HTTP

HTTP-Header

- Zusätzliche Metadaten, die mit Request/Response gesendet werden
- Key-Value-Paare mit Informationen über die Anfrage/Antwort
- Übertragen Informationen über Inhalt, Authentifizierung, Caching, etc.

Beispiel-Request mit Headern:

```
GET /api/users/123 HTTP/1.1
Host: api.example.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json
Accept: application/json
User-Agent: Mozilla/5.0
```

* Siehe auch https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers

HTTP

HTTP-Header

Häufige Header:

- *Content-Type*: Format der Daten (z.B. application/json)
- *Authorization*: Authentifizierungsinformationen
- *Accept*: Gewünschtes Antwortformat
- *User-Agent*: Information über Client-Software
- *Content-Length*: Größe des Bodys in Bytes

* Siehe auch https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers

REST-APIs

Definition API (Application Programming Interface)

- Schnittstelle für Kommunikation zwischen Softwarekomponenten
- Definiert Regeln und Protokolle, wie Anfragen und Antworten aussehen
- Ermöglicht standardisierte Interaktion zwischen Systemen

Beispiel: Abrufen einer Wettervorhersage in einer App

REST-APIs

Representational State Transfer (REST)

REST API = API, die HTTP als Transportprotokoll benutzt

Prinzipien

1. **Ressourcen-orientiert**: Alles ist eine Ressource (URI)
2. **Zustandslos**: Jede Anfrage ist unabhängig
3. **Einheitliche Schnittstelle**: Standard-HTTP-Methoden
4. **Client-Server-Architektur**: Trennung der Verantwortlichkeiten
5. **Cacheability**: Antworten können gecacht werden

REST-APIs

JavaScript Object Notation (JSON)

- **Textbasiertes** Datenformat für **strukturierte** Daten
- **Leicht lesbar** für Menschen und Maschinen
- Häufig Standard-Datenaustauschformat für REST-APIs
- **Sprach-unabhängig** (trotz JavaScript im Namen)
- **Datentypen:** String, Number, Boolean, Array, Object, null

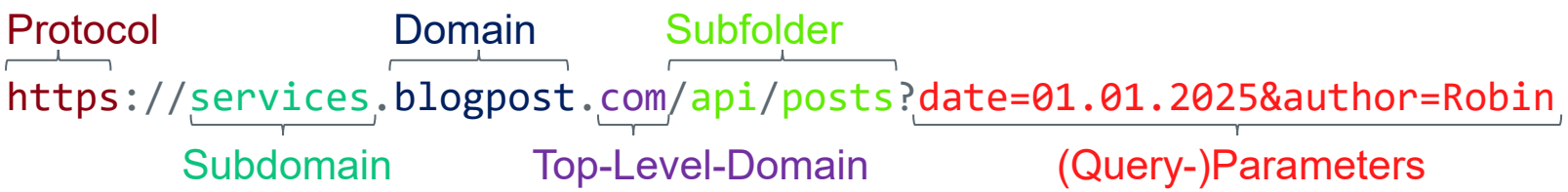
Beispiel: JSON für Blog-Beitrag

```
{
  "id": 123,
  "title": "My first blog entry",
  "text": "This is my first blog entry",
  "views": 42,
  "isActive": true,
  "comments": [
     {
       "id": 50,
       "postId": 123,
       "text": "Well done!"
     }
   ],
   "author": {
     "id": 1,
     "firstName": "Robin",
     "lastName": "Windey"
   }
}
```

REST-APIs

API-Design

URL-Aufbau



Beispielhafte URL-Struktur

`GET`	`/api/posts`	→ Alle Blog-Beiträge abrufen
`GET`	`/api/posts/123`	→ Beitrag mit ID 123 abrufen
`POST`	`/api/posts`	→ Neuen Blog-Beitrag erstellen
`PUT`	`/api/posts/123`	→ Beitrag 123 aktualisieren
`DELETE`	`/api/posts/123`	→ Beitrag 123 löschen

`GET /api/posts?date=01.01.2025&author=Robin`

→ Alle Beiträge von Robin vom 01.01.2025 abrufen (Filter als Query-Parameter)

REST-APIs

API-Design

Best Practices

- Substantive verwenden (nicht Verben)
- Plural für Ressourcen
- Hierarchien durch "/" trennen
- Versionierung: /api/v1/posts, /api/v2/posts

Aufgabe



REST API Interaktion mit cURL

- Starte "json-server" als Docker Container:
  `docker run --name json-server -d -p 3000:3000 ghcr.io/r0wi/dhbw-verteilte-systeme/json-server:latest`
- Inspiziere die Logs: `docker logs json-server`
- Nutze die Doku: https://github.com/typicode/json-server?tab=readme-ov-file#params

Beispiel: `curl "http://localhost:3000/posts"` → Ruft alle Blogbeiträge ab

1. Rufe alle Kommentare ab
2. Rufe Blogbeitrag mit Id "1" ab
3. Rufe Blogbeitrag mit Id "1" ab und bette alle Kommentare des Beitrags ein
4. Rufe Blogbeiträge mit mehr als 150 views ab
5. Rufe den Blogbeitrag mit Titel "The answer to all questions" ab und bette die Kommentare ein (Achtung: Leerzeichen brauchen URL Encoding)

REST-APIs

OpenAPI Specification (OAS)

Was ist OAS?

- Standard zur Beschreibung von REST-APIs
- Maschinenlesbar (YAML/JSON)
- Ermöglicht automatische Dokumentation
- Tools: Swagger UI, Redoc

Vorteile

- Klare Dokumentation
- Code-Generierung möglich
- Interaktive API-Tests
- Vertragsdefinition zwischen Frontend/Backend

* Siehe auch https://swagger.io/specification/

Beispiel: OAS für Blog-Beitrag API

```yaml
openapi: 3.0.3
info:
  title: Blog Posts API
  version: "1.0"
paths:
  /api/posts:
    get:
    # ...
    post:
    # ...
  /api/posts/{id}:
    parameters:
      - name: id
        in: path
        required: true
        schema:
          type: integer
    get:
      summary: Beitrag abrufen
      responses:
        "200":
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Post'
        "404": # ...
components:
  schemas:
    Post:
      type: object
      properties:
        id:
          type: string
          example: "123"
      # ...
      required:
        - title
```

*Vollständige Datei: siehe Moodle

Aufgabe



REST API Interaktion mit Swagger

- Füge die OAS YAML Datei in https://editor.swagger.io/ ein

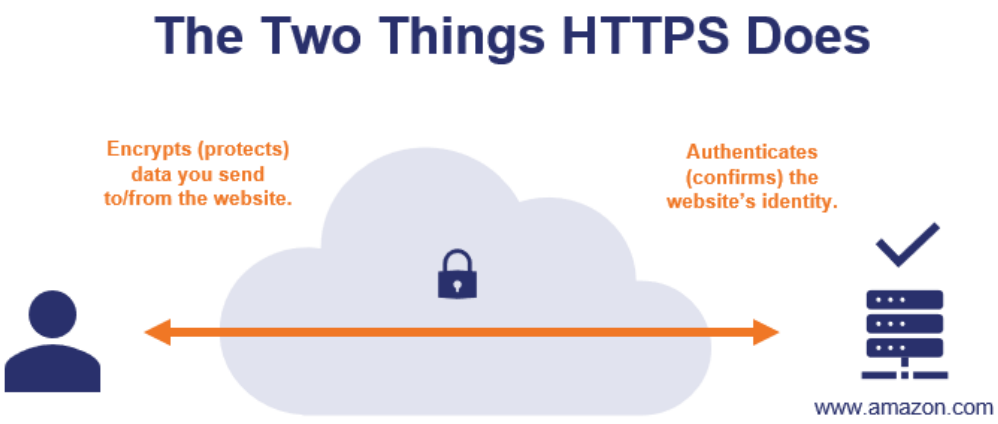
1. Rufe alle Blogposts inklusive aller Kommentare ab
2. Lege einen neuen Blogpost mit Id 43 und beliebigem Inhalt an
3. Lege einen neuen Kommentar zu Blogpost 43 mit Kommentar-Id 53 an
4. Ändere den Inhalt des neu angelegten Kommentars mit Id 53 zu "Neuer Kommentarinhalt"
5. Rufe alle Blogposts mit Kommentaren als "Paginated Result" mit Page Size 2 ab
6. Lösche den Blogpost mit Id 42 inklusive seiner Kommentare (mit einem API-Call). Vergewissere dich, dass der Post auch wirklich gelöscht wurde

* https://gotapi.com/wp-content/uploads/2023/10/1_R36nHDnQ9i7vizbSJqTb1g.png

HTTPS & Sicherheit

HTTPS = HTTP + TLS/SSL

- Verschlüsselte Übertragung
- Authentifizierung des Servers
- Schutz vor Man-in-the-Middle-Angriffen



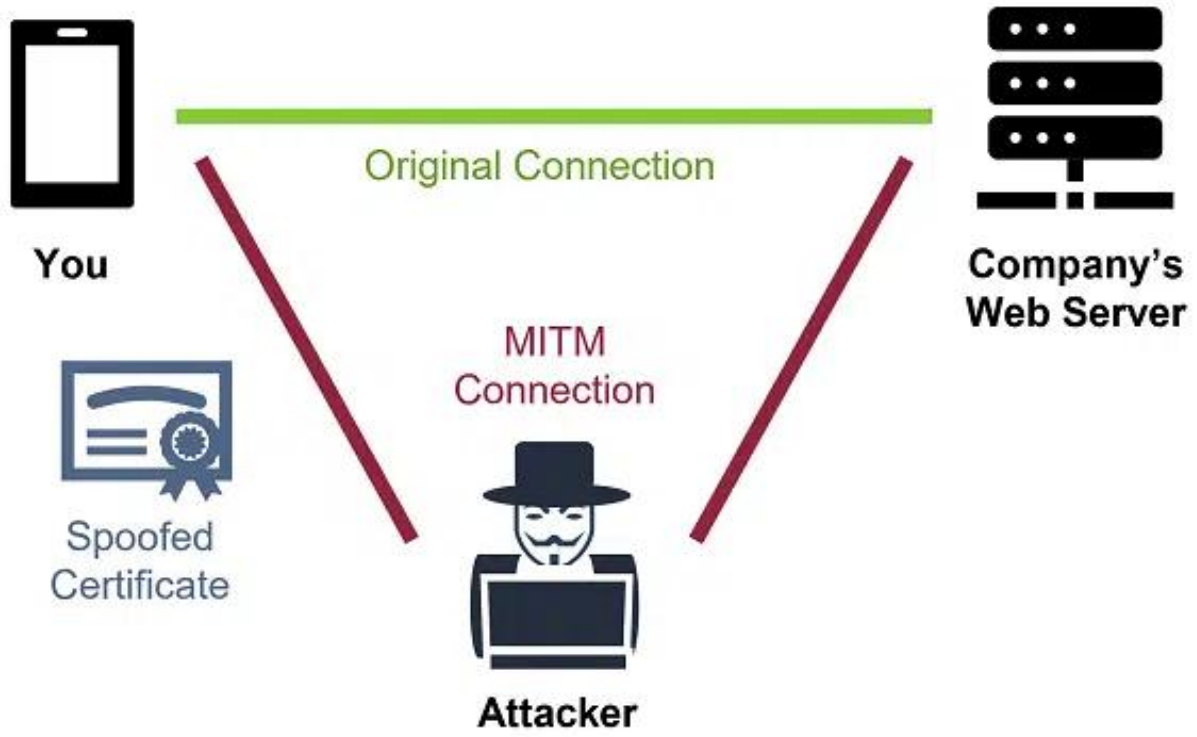
Zertifikate

- X.509 Zertifikate
- Certificate Authorities (CA)

* https://www.thesslstore.com/blog/wp-content/uploads/2020/10/how-HTTPS-works-the-two-things-HTTPS-does.png

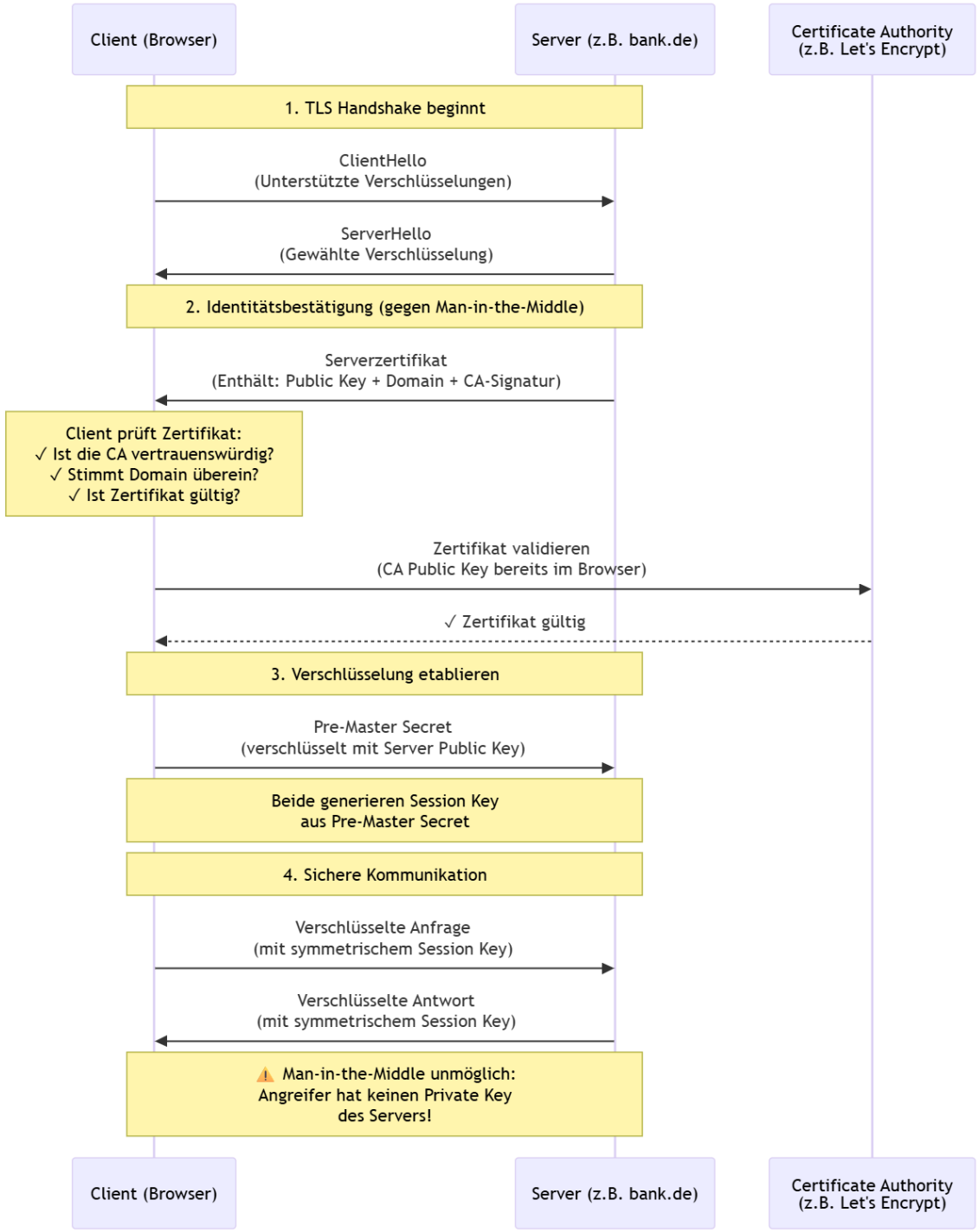
HTTPS & Sicherheit

Man-in-the-Middle-Attack (MITM)



* https://www.enea.com/wp-content/uploads/2022/08/MITM_Attacks.png

Flow: TLS-Verbindungsaufbau



HTTPS & Sicherheit

Authentifizierung vs. **Autorisierung**

- **Authentifizierung**:
  "Wer bist du?"
  Identitätsnachweis (Login mit Username/Passwort)
- **Autorisierung**:
  "Was darfst du?"
  Berechtigungsprüfung (Zugriff auf Ressourcen)

Beispiel:

- Authentifizierung = Hotelzimmer-Karte bekommen
- Autorisierung = Nur eigenes Zimmer öffnen

HTTPS & Sicherheit

Basic Auth

- `<Username:Password>` in Base64 kodiert
- Im Authorization-Header (Client): `Authorization: Basic dXNlcjpwYXNz`
- Unsicher ohne HTTPS
- Einfach, aber veraltet

HTTPS & Sicherheit

Token-basiert (z.B. JWT - JSON Web Token)

- Zustandslos (stateless) und selbst-enthaltend (self-contained)
- Wird im Authorization-Header mit "Bearer"-Schema gesendet
- Struktur: `<Header>.<Payload>.<Signature>`
  `Authorization: Bearer eyJhbGciOiJIU.eyJzdWIiOiJSMFdp.O14K8aFIHdRk`
- Enthält Benutzerinformationen (Claims) und Ablaufzeit
- Wird für **Authentifizierung** UND **Autorisierung** verwendet
- **Access Token:** Signiertes Token, das der Client dem Resource Server vorlegt, um Zugriff (Scopes) nachzuweisen.
- **Refresh Token (optional):** Token zum Anfordern neuer Access Tokens ohne erneute Benutzerinteraktion.

Aufgabe



Nutze https://www.jwt.io/ um folgende Fragen über das unten angehängte JWT zu beantworten:

1. Welcher Verschlüsselungs- bzw. Signatur-Algorithmus wurde für das JWT verwendet?
2. Wie lautet die E-Mail des Benutzers?
3. In welcher Rolle darf der Benutzer die Applikation verwenden?
4. Wann wurde das Token ausgestellt und wann läuft es ab?

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJSMFdpIiwibmFtZSI6IlJvYmluIFdpbm
RleSIsImVtYWlsIjoicm8ud2luZGV5QGdtYWlsLmNvbSIsInJvbGUiOiJhZG1pbiIsImV4cCI6MTczN
TY4OTYwMCwiaWF0IjoxNzM1Njg2MDAwfQ.O14K8aFIHd8xH09Qrenkbel82jrWrjAEQwFmHneUXRk
```

Autorisierung

OAuth 2.0 Delegated Authorization

Was ist OAuth 2.0?

- Framework für sichere Autorisierung
- Ermöglicht Drittanbieter-Zugriff ohne Passwort-Weitergabe
- Beispiel: "Mit Google anmelden", "Mit Facebook anmelden"
- **Kombiniert**: Authentifizierung (wer) + Autorisierung (was)
- **Aber**: Spezifikation schreibt lediglich Regeln für Autorisierung vor (nicht für Authentifizierung!)

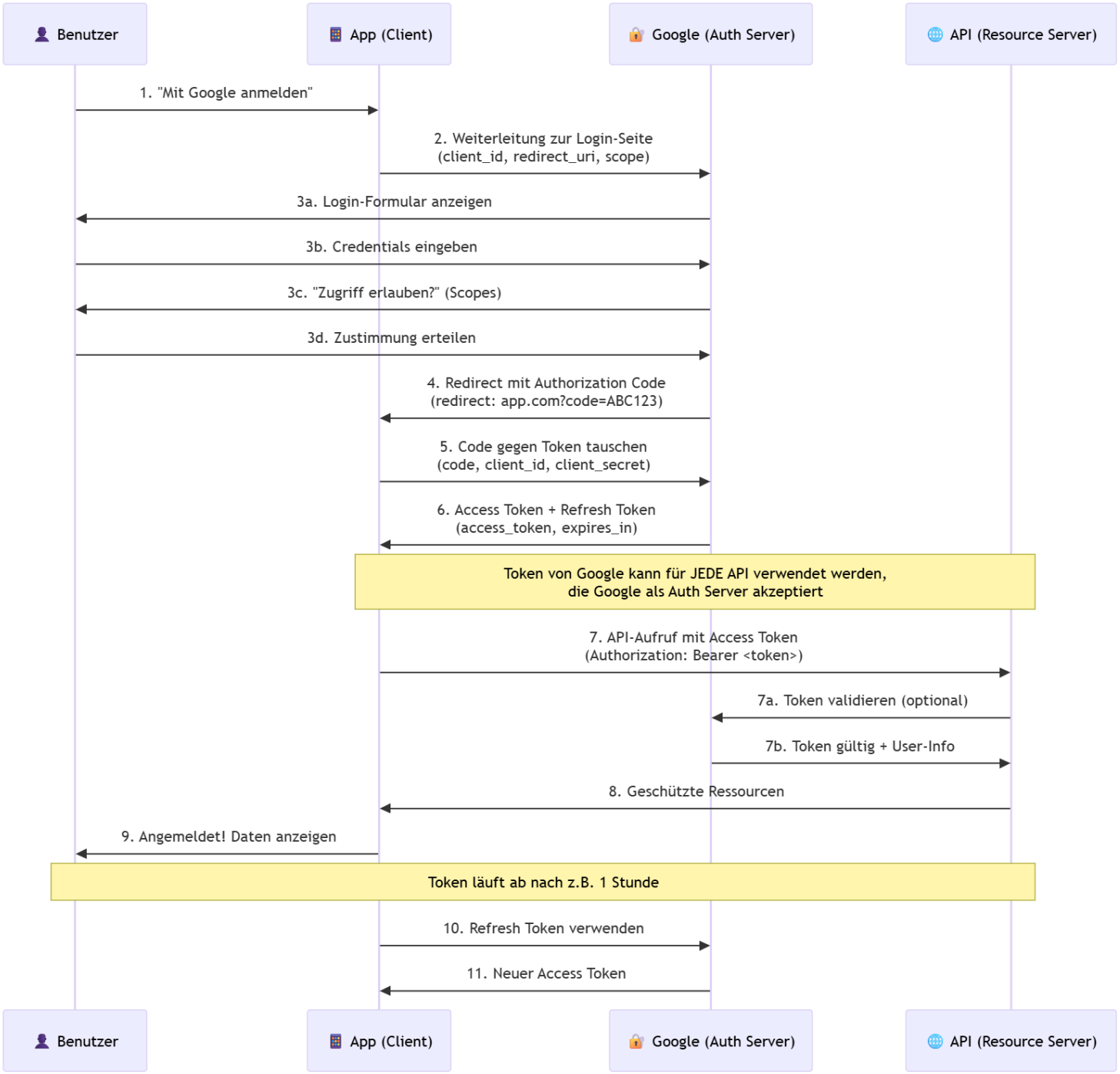
⚠ Es gibt verschiedene OAuth 2.0 Flows für verschiedene Anwendungsfälle. Siehe https://auth0.com/docs/get-started/authentication-and-authorization-flow/which-oauth-2-0-flow-should-i-use

Autorisierung

OAuth 2.0 Terminologie

- **Resource Owner (Ressourcen-Eigentümer):** Entität, die Zugriff auf eine geschützte Ressource gewähren kann – typischerweise der Endbenutzer.
- **Client:** Anwendung, die im Auftrag des Resource Owners Zugriff auf eine geschützte Ressource anfordert (z. B. Web-App, Mobile App).
- **Resource Server:** Server, der die geschützten Ressourcen (APIs, Daten) bereitstellt und Access Tokens prüft (die Ziel-API).
- **Authorization Server (Autorisierungsserver):** Server, der den Resource Owner authentifiziert, Zustimmung (Consent) einholt und Access Tokens (und ggf. Refresh Tokens) ausstellt (z. B. Auth0, Google, …).
- **User Agent:** Vermittelnde Instanz, über die der Resource Owner mit dem Client interagiert (Browser, native App WebView).
- **Scope:** Begrenzung des Zugriffsumfangs (z. B. read:orders, write:orders).
- **Consent:** Explizite Zustimmung des Resource Owners zu angeforderten Scopes.

OAuth 2.0: Authorization Code Flow



*Siehe auch https://auth0.com/docs/get-started/authentication-and-authorization-flow/authorization-code-flow

DHBW

MQTT

Message Queuing Telemetry Transport

- Publish & Subscribe – kurz Pub/Sub
- Einsatzgebiet vorwiegend IoT (abgetrennte Industrienetze bis hin zu Home Automation)
- Beispiel: Newsletter (Post/E-Mail)

- MQTT **Broker**: "Handelt" mit Nachrichten
- MQTT Clients
  - **Publisher**: Veröffentlicht Nachrichten
  - **Subscriber**: Abonniert Nachrichtenthemen
- Organisiert in **Topics** (Themen) nach Schema:

```
haus/erdgeschoss/wohnzimmer/temperatur
```

MQTT

Arbeiten mit **MQTT**

- Neue Nachrichten auf einem Topic werden zu **allen** Subscribern verteilt
- Subscriber können dank Wildcards beliebige Topic-Kombinationen abonnieren
  - Wildcard für eine Ebene: **+**
  - Wildcard für alle darunterliegenden Ebenen: #
  - Bspw: `haus/erdgeschoss/+/temperatur`
    `haus/erdgeschoss/wohnzimmer/#`

QoS in MQTT

Gültig pro Nachricht, Client & Verbindung

- Default: QoS Level 0 - "At most once" / "Fire and forget"
  - Nachricht wird abgesendet und an Subscriber gesendet
  - Keine weiteren Maßnahmen oder Nachverfolgung
  - Kein Overhead
- QoS Level 1 - "At least once"
  - Nachricht wird abgesendet und wartet auf **PUBACK** vom Broker
  - Wenn kein **PUBACK** erhalten wird, wird nochmals gesendet
  - Wenig Overhead
- QoS Level 2 - "Exactly once"
  - Vollwertiger 4-stufiger Handshake
  - Stellt sicher, dass eine Nachricht genau einmal zugestellt wird
  - Viel Overhead



MQTT
Diskussion

- Müssen sich Publisher und Subscriber kennen? Warum oder warum nicht?
- Wie müssen verarbeitende Programme auf die einzelnen QoS Level abgestimmt werden?
- Was passiert, wenn Publisher und Subscriber anderwertige QoS Level definieren?
- Was muss bei QoS Level 1 & 2 zwingend vorhanden sein?

Docker compose

- "Mini"-IaC (Infrastructure as Code)
- Problem: Konfigurationen für Docker führen zu langen Befehlen, viele Container führen zu vielen Befehlen
- Lösung: Wir deklarieren unsere Soll-Umgebung mit einer Datei:
  `docker-compose.yaml`

- Compose hält Konfigurationen, Netzwerke, Umgebungsvariablen, handhabt Dependencies und kann mehrere Container auf einmal starten
- Compose erleichtert Netzwerke: By default wird ein gemeinsames Netzwerk erstellt, in dem wir unsere Container mit ihrem Service-Namen ansprechen können
- Workflow: In einem Verzeichnis mit `docker-compose.yaml` führen wir aus:
  ```
  docker compose up
  ```

Docker compose

```yaml
services:
  prometheus:
    image: prom/prometheus
    container_name: prometheus
    command:
      - '--config.file=/etc/prometheus/prometheus.yml'
    ports:
      - 9090:9090
    restart: unless-stopped
    volumes:
      - ./prometheus:/etc/prometheus
      - prom_data:/prometheus
  grafana:
    image: grafana/grafana
    container_name: grafana
    ports:
      - 3000:3000
    restart: unless-stopped
    environment:
      - GF_SECURITY_ADMIN_USER=admin
      - GF_SECURITY_ADMIN_PASSWORD=grafana
    volumes:
      - ./grafana:/etc/grafana/provisioning/datasources
volumes:
  prom_data:
```


https://github.com/docker/awesome-compose/blob/master/prometheus-grafana/compose.yaml

Aufgabe



- Analysiert das heruntergeladene Compose File:

- Wie ist es aufgebaut? Was fällt euch auf? Wie viele Container lassen sich starten? Was passiert dabei im Hintergrund?
- Auf welches Topic "hört" der Subscriber?

- Schaut außerdem kurz auf die Konfiguration unter `mosquitto/mosquitto.conf`:

- Warum ist diese Config für einen produktiven Einsatz schlecht?
- Welche Alternativen haben wir? Doku: https://mosquitto.org/man/mosquitto-conf-5.html

Praxis: MQTT

Spielen

- Erstellt zunächst einen Subsciber:
- Im gleichen Ordner wie das `docker-compose.yaml` ausführen:
- `docker compose run subscriber`

- Erstellt dann einen Publisher:
- In einer neuen Kommandozeile im gleichen Verzeichnis:
- `docker compose run publisher`

- Sendet Nachrichten:
- Im Publisher-Fenster:
- `mosquitto_pub -h mosquitto -t test/topic -m "<NACHRICHT>"`

Praxis: MQTT

Spielen

- Öffnet einen zweiten Subscriber (neue Command Line mit gleichem Befehl):
- `docker compose run subscriber`

- Ändert das Topic
- Erstellt einen weiteren Subscriber, der auf ein anderes Topic hört
- Probiert Wildcards aus
- ???
- Probiert Authentifizierung aus, break it, fix it, have fun with it

Praxis: MQTT

Python



- Schreibt ein kleines Python Script, das mit eurem aufgesetzen MQTT Broker interagieren kann
- Vereinfachung: Nur lesender Zugriff
- Nutzt bestenfall virtual environments in python

- `pip install paho-mqtt`

- For reference:
  > https://www.hivemq.com/blog/mqtt-client-library-paho-python/